Ne rien inscrire dans ce cadre

Prénom .....

Nom .....

Promotion .....

Groupe .....

Promotions : L2, L2-BN, L2-BDX, L2-P

Module : Programmation Orientée Objet en Java

18/05/2024 de 12H40 à 14H30

Examen sur table

Sujet rédigé par : Madame Cherifa Ben Khelil

Supports autorisés :

Calculatrice autorisée : **non**

Documents autorisés : **non**

Traducteur électronique : **non**

Dictionnaire : **non**

Consigne :

Merci de restituer : **le sujet avec vos réponses**

Rappel :

- Tous les appareils électroniques (téléphones portables, ordinateurs, tablettes, montres connectées, accès à internet …) doivent être éteints et rangés.
- Il est interdit de communiquer.
- Toute fraude ou tentative de fraude fera l'objet d'un rapport de la part du surveillant et sera sanctionnée par la note zéro, assortie d'une convocation devant le conseil de discipline. Aucune contestation ne sera possible. Tous les documents et supports utilisés frauduleusement devront être remis au surveillant.
- Aucune sortie de la salle d'examen ne sera autorisée avant la moitié de la durée de l'épreuve.

Ne rien inscrire dans ce cadre

S'il vous plaît, n'écrivez pas au crayon !!

La note sera corrigée sur une échelle de 30 points, mais sera ramenée à 20.

Partie I : Compréhension (4 points) - Temps recommandé : 20 minutes

Exercice 1 (2 points) :

Cochez **la** bonne réponse :

Question 1
Quel principe de la programmation orientée objet permet de regrouper des données et des méthodes qui leur sont associées ? ☐ Surcharge ☐ Héritage ☐ Encapsulation ☐ Redéfinition ☐ Polymorphisme
Question 2
Quel mécanisme permet à une classe d'avoir plusieurs méthodes avec le même nom mais des signatures différentes ? ☐ Surcharge ☐ Héritage ☐ Encapsulation ☐ Redéfinition ☐ Polymorphisme
Question 3
Quel mécanisme permet à une sous-classe de fournir une implémentation spécifique d'une méthode héritée de sa classe parente ? ☐ Surcharge ☐ Héritage ☐ Encapsulation ☐ Redéfinition ☐ Polymorphisme
Question 4
Quel est le terme utilisé pour désigner une classe qui ne peut pas être instanciée et sert de modèle pour d'autres classes ? ☐ La super classe ☐ La classe Object ☐ La classe abstraite ☐ Le classe concrète

Question 5
L'utilisation de @Override lors de la redéfinition de méthodes héritées d'une classe parente ou lors de l'implémentation de méthodes d'une interface est-elle obligatoire ? ☐ Oui ☐ Non ☐ Obligatoire dans certains cas
Question 6
Quel mot-clé est utilisé pour déclarer une constante (une variable constante) en Java? ☐ static ☐ final ☐ const ☐ constant ☐ abstract
Question 7
Quel mot-clé est utilisé pour permettre l'accès aux méthodes d'une classe sans instancier d'objet en Java? ☐ static ☐ final ☐ instant ☐ new ☐ abstract
Question 8
Quel mot-clé est utilisé pour spécifier qu'une méthode d'une classe devra être redéfinie dans les classes filles en Java? ☐ static ☐ protected ☐ extends ☐ abstract

Exercice 2 (2 points) :

Considérons le code suivant :

```
public interface Geometrie {
    int aire();
    double perimetre();
}

public class Cercle implements Geometrie{
    private double rayon;

    @Override
    public double aire() {
        return Math.PI * rayon * rayon;
    }

    @Override
    public double perimetre() {
        return 2 * Math.PI * rayon;
    }
}
```

Le code ci-dessus comporte **deux problèmes**. Identifiez et expliquez brièvement votre choix pour chacun, puis suggérez une solution en code pour les résoudre.

N°	Identification et explication	Solution (code)
1		
2		

Partie II : Exécution (5.5 points) - Temps recommandé : 30 minutes

Soit le code ci-dessous

```
public class A {
    private int vA;
    public A() { vA = -1;}
    public A(int vA){ this.vA = vA; }
    public double getValeur(){ return vA; }
    public double somme (A a){
        return getValeur() + a.getValeur(); }
    public static double somme (A a1, A a2){
        return a1.somme(a2);}
```

```
    public String toString(){
        return "A(" +vA + ")";}}
```

```
public class B extends A {
    private double vB;
    public B() { vB = -2;}
    public B(int vA, double vB) {
        super(vA);
        this.vB = vB; }
    @Override
    public double getValeur() { return vB; }
    @Override
    public double somme(A a) {
        return somme(this, a); }}
```

```
public class C extends A {
    private int vC;
    public C() { vC = -3;}
    public C(int vC) {
        super(vC + 1);
        this.vC = vC; }
    public C(int vA, int vC) {
        super(vA);
        this.vC = vC; }
    public int getvC() { return vC; }
    @Override
    public double getValeur() { return vC; }
    public double somme(C c) {
        return super.somme(c) + 1;}
    @Override
    public String toString() {
        return "C<"+ super.toString()+ ">(" + vC + ")";}}
```

```
public class D extends C {
    private String vD;
    public D() { this("-4"); }
    public D(String vD) {
        super(Integer.parseInt(vD));
        this.vD=vD; }
```

```
    public D(int vA, String vD) {
        super(vA, Integer.parseInt(vD));
        this.vD=vD;}
    @Override
    public double somme (A a){
        if ( a instanceof D)
            return getvC()*((C)a).getvC();
        return super.somme(a);}
    @Override
    public String toString(){
        return "D<"+super.toString()+ ">(\"" + vD + "\")";
}}
```

Rappel : l'opérateur "**instanceof**" permet de vérifier si un objet est une instance d'une classe spécifique. Il renvoie *true* si l'objet est une instance de la classe spécifiée ou de l'une de ses sous-classes, sinon il renvoie *false*.

Question 1 (1 point) : Quel affichage résulterait de l'exécution des instructions suivantes ?

```
A a = new C( 500, 12) ;
System.out.println (a);
D d=new D( 500, "12");
System.out.println (d);
```

Réponse :

……

……

……

Question 2 (4.5 points) : Considérons les initialisations suivantes :

```
A a1=new A();
A a2=new A();
B b1=new B();
B b2=new B();
C c= new C();
D d1=new D();
D d2=new D();
```

Déterminez le résultat de chaque instruction.

NB : il est possible que certaines instructions ne produisent pas de sortie ou entraînent des erreurs lors de leur exécution.

N°	Instruction	Réponse
1	`System.out.println(A.somme(a1, a2));`	
2	`System.out.println(a1.somme(d1));`	
3	`System.out.println(B.somme(b1 , b2));`	
4	`System.out.println(b2.somme(c));`	
5	`System.out.println(C.somme(d1 , d2));`	
6	`System.out.println(c.somme(b1));`	
7	`System.out.println(C.somme(a2,b2));`	
8	`System.out.println(D.somme(b1 , b2));`	
9	`System.out.println(d1.somme(a1));`	

Partie III : Codage (20.5 points) - Temps recommandé : 60 minutes

Cette partie comporte **3 questions** (5.5, 3.5 et 11.5 points), **chacune comprenant 2 sous-questions**. Bien que les questions de cette partie soient liées, il est **possible de résoudre certaines réponses** de code partiel en suivant les directives spécifiques de chaque question.

Considérons le code suivant, composé d'une classe **Main** qui inclut trois méthodes :

```
import java.util.Scanner;

public class Main {
    // Méthode pour vérifier si les longueurs forment un triangle
    public static boolean estTriangle(int a, int b, int c) {
        return (a + b > c) && (a + c > b) && (b + c > a);
    }

    // Méthode pour vérifier si le triangle est rectangle
    public static boolean estTriangleRectangle(int a, int b, int c) {
        return (a * a + b * b == c * c) || (a * a + c * c == b * b) || (b * b + c * c
== a * a);
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Saisie des longueurs a, b, c
        System.out.println("Entrez les longueurs a, b, c du triangle :");
        System.out.print("a : ");
        int a = scanner.nextInt();
        System.out.print("b : ");
        int b = scanner.nextInt();
        System.out.print("c : ");
        int c = scanner.nextInt();

        // Vérification si les longueurs peuvent former un triangle
        if (estTriangle(a, b, c)) {
            System.out.println("Ces longueurs peuvent former un triangle.");
```

```java
            // Vérification du type de triangle
            if (a == b && b == c) {
                System.out.println("Type de triangle : Équilatéral.");
            } else if (a == b || a == c || b == c) {
                System.out.println("Type de triangle : Isocèle.");
            } else {
                System.out.println("Type de triangle : Quelconque.");
            }

            // Vérification si le triangle est rectangle
            if (estTriangleRectangle(a, b, c)) {
                System.out.println("Le triangle est rectangle.");
            }
        } else {
            System.out.println("Ces longueurs ne peuvent pas former un triangle.");
}}}
```

Le programme fonctionne comme suit :

i. Il demande à l'utilisateur de saisir les longueurs des côtés du triangle.

ii. Il utilise deux méthodes **estTriangle()** et **estTriangleRectangle()** pour vérifier si les longueurs saisies peuvent former un triangle et si le triangle est un triangle rectangle, respectivement.

iii. En fonction des résultats de ces vérifications, le programme indiquera si les longueurs saisies peuvent former un triangle et affichera son type (exemple **1** et **2**). S'il s'agit d'un triangle rectangle, cette information sera également fournie (exemple **2**). Dans le cas contraire, il affichera un message précisant que les longueurs ne permettent pas de former un triangle (exemple **3**).

Exemples d'exécutions :

1
```
Entrez les longueurs a, b, c du triangle :
a : 5
b : 5
c : 4
Ces longueurs peuvent former un triangle.
Type de triangle : Isocèle.
```

2
```
Entrez les longueurs a, b, c du triangle :
a : 4
b : 5
c : 3
Ces longueurs peuvent former un triangle.
Type de triangle : Quelconque.
Le triangle est rectangle.
```

3
```
Entrez les longueurs a, b, c du triangle :
a : 5
b : 5
c : 26
Ces longueurs ne peuvent pas former un triangle.
```

Question 1 (5.5 points) :

1.1. Nous voulons modifier la classe **Main** de manière à ce qu'elle ne contienne plus que la méthode ***main***. Pour ce faire, créez une nouvelle classe supplémentaire appelée **Triangle**, qui représente un triangle géométrique. Cette classe **(Triangle)** doit contenir les éléments suivants :

- Trois attributs privés de type entier **a**, **b** et **c** représentant les longueurs des côtés du triangle.
- Un constructeur pour initialiser ces trois attributs.
- Trois méthodes publiques d'accès en ***lecture***, chacune correspondant à l'un des trois attributs.
- La méthode **estTriangle()** qui retourne un booléen indiquant si les longueurs données du triangle courant peuvent former un triangle.
- La méthode **estTriangleRectangle()** qui retourne un booléen indiquant si le triangle courant est un triangle rectangle.

Réponse 1.1 : Définition de la classe Triangle

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

1.2. Complétez la classe **Main**, ci-dessous, qui désormais ne contient qu'une seule méthode ***main***. Assurez-vous que le code proposé génère le même résultat que celui obtenu avec la version précédente.

Réponse 1.2 :

```
import java.util.Scanner;
public class Main {
 public static void main(String[] args) {
   Scanner scanner = new Scanner(System.in);
   System.out.println("Entrez les longueurs a, b, c du triangle :");
   System.out.print("a : ");
   int a = scanner.nextInt();
   System.out.print("b : ");
   int b = scanner.nextInt();
   System.out.print("c : ");
   int c = scanner.nextInt();
   // Instanciation d'un triangle (à compléter)

   ..................................................................................
```

```
    // Vérification si le triangle peut être formé (à compléter)
    if ..................................................................... {
        System.out.println("Ces longueurs peuvent former un triangle.");
        // Affichage du type de triangle (à compléter)
    if..................................................................... {
         System.out.println("Type de triangle : Équilatéral."); }
    else if ............................................................... {
         System.out.println("Type de triangle : Isocèle.");
    } else {
         System.out.println("Type de triangle : Quelconque."); }
    // Vérification si le triangle est rectangle (à compléter)
    if ..................................................................... {
        System.out.println("Le triangle est rectangle."); }
} else {
     System.out.println("Ces longueurs ne peuvent pas former un triangle.");
}}}
```

Question 2 (3.5 points):

Définir deux nouvelles méthodes dans la classe **Triangle** :

- La méthode **getTypeTriangle()** : elle utilise les longueurs des côtés du triangle (a, b et c) pour déterminer son type. Cette méthode renvoie une chaîne de caractères indiquant le type de triangle, qui peut être "*Équilatéral.* ", "*Isocèle.*" ou "*Quelconque.* ".
- La méthode **afficherInfos() :** Cette méthode affiche les informations sur le triangle : Elle commence par vérifier si les longueurs des côtés peuvent effectivement former un triangle en utilisant la méthode **estTriangle()**. Ensuite, elle utilise la méthode **getTypeTriangle()** pour obtenir et afficher le type de triangle. Si le triangle est un triangle rectangle (en utilisant la méthode **estTriangleRectangle()**), elle affiche également un message indiquant que le triangle est rectangle. Si les longueurs des côtés ne peuvent pas former un triangle, elle affiche le message suivant : *"Ces longueurs ne peuvent pas former un triangle"*.

Réponse 2.1: Veuillez fournir uniquement le code des deux méthodes requises sans réécrire l'intégralité de la classe Triangle.

```
// Définition de la méthode getTypeTriangle()

……
……
……
……
……
……
……
……
……
……
……
……
……
……
```

```
// Définition de la méthode afficherInfos()

……
……
……
……
……
……
……
……
```

……

……

……

……

……

……

2.2. Finalisez la classe **Main** pour créer un triangle et l'afficher, en tenant compte des deux nouvelles méthodes ajoutées à la classe **Triangle**. Assurez-vous que le code proposé génère le même résultat que celui obtenu avec la version précédente.

Réponse 2.2 :

```
import java.util.Scanner;
public class Main {
 public static void main(String[] args) {
   Scanner scanner = new Scanner(System.in);
   System.out.println("Entrez les longueurs a, b, c du triangle :");
   System.out.print("a : ");
   int a = scanner.nextInt();
   System.out.print("b : ");
   int b = scanner.nextInt();
   System.out.print("c : ");
   int c = scanner.nextInt();

    // Création d'un triangle (à compléter)

   ……

   // Affichage des informations sur le triangle (à compléter)

   ……



 }}
```

Question 3 (11.5 points) :

3.1. Utilisez les concepts d'héritage, d'abstraction et de polymorphisme pour transformer la classe **Triangle** et concevoir une hiérarchie d'héritage efficace comprenant les quatre classes suivantes : **Triangle**, **TriangleEquilateral**, **TriangleIsocele** et **TriangleQuelconque** :

- Chaque classe doit posséder un constructeur destiné à initialiser les attributs a, b et c, qui représentent respectivement les longueurs des côtés du triangle. Cependant, seuls les objets de types **TriangleEquilateral**, **TriangleIsocele** et **TriangleQuelconque** pourront être instanciés.
- Intégrez les quatre méthodes suivantes dans cette hiérarchie de classes, en les définissant ou en les redéfinissant selon les besoins spécifiques de chaque classe. Notez que certaines de ces méthodes seront implémentées dans une/des classe(s), tandis que d'autres non :
  - La méthode **estTriangle()** pour vérifier si les longueurs peuvent former un triangle.
  - La méthode **estTriangleRectangle()** pour vérifier si le triangle est rectangle.
  - La méthode **getTypeTriangle()** pour retourner le type de triangle, qui peut être "*Équilatéral.*", "*Isocèle.*" ou "*Quelconque.* ".
  - La redéfinition de la méthode **toString()** afin de remplacer la méthode **afficherInfos()** : Cette méthode servira à afficher les informations sur le triangle. Elle doit retourner le même résultat que celui affiché par la méthode ***afficherInfos()*** de la ***question 2.2***.

NB : Pour cette question, il n'est pas demandé de définir des accesseurs et des mutateurs, y compris pour la classe **Triangle**, donc ne pas le faire.

Réponse 3.1:

```
// Définition de la classe Triangle

……
……
……
……
……
……
……
……
……
……
……
……
```

```
// Définition de la classe TriangleEquilateral
```

```
// Définition de la classe TriangleIsocele
```

```
// Définition de la classe TriangleQuelconque
```

3.2. Proposez une révision de la classe **Main** intégrant la méthode ***main***. Le code modifié doit inclure les nouvelles classes et générer le même résultat que celui obtenu avec la version précédente :

- Demandez à l'utilisateur de saisir les longueurs des côtés a, b et c du triangle.
- Utilisez des <u>conditions</u> pour déterminer le type de triangle et instancier l'objet en fonction des longueurs saisies par l'utilisateur.
- Affichez les informations sur le triangle.

Rappel des exemples d'exécutions attendues :

1 Entrez les longueurs a, b, c du triangle : a : 5 b : 5 c : 4 Ces longueurs peuvent former un triangle. Type de triangle : Isocèle.	**2** Entrez les longueurs a, b, c du triangle : a : 4 b : 5 c : 3 Ces longueurs peuvent former un triangle. Type de triangle : Quelconque. Le triangle est rectangle.

Réponse 3.2 :

```java
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Entrez les longueurs a, b, c du triangle :");
        System.out.print("a : ");
        int a = scanner.nextInt();
        System.out.print("b : ");
        int b = scanner.nextInt();
        System.out.print("c : ");
        int c = scanner.nextInt();

// suite à compléter

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
    }
}
```